snapdeal

voice_kam_bot_020bf354.plan.md

napdeal_voice_kam_bot_020bf354.plan.md

Claude Sonnet 4.6 Medium

Snapdeal Voice KAM Bot — Complete Build Plan (Gemini + Web Speech API)

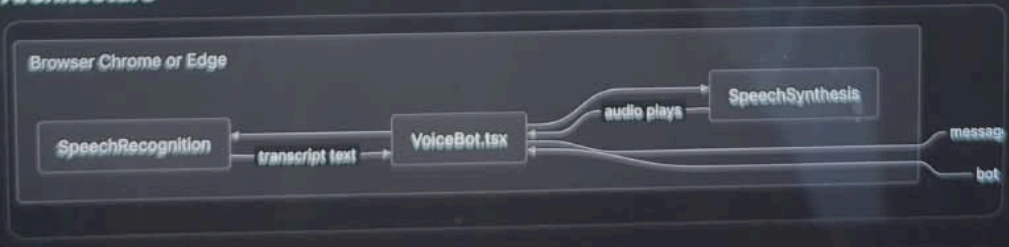
Key Tech Decision

Component	What we use	Cost
LLM / Conversation	Gemini 1.5 Flash via `@google/generative-ai`	Free tier (1500 req/day)
Speech-to-Text (STT)	Browser **Web Speech API** (`SpeechRecognition`)	Free, zero API calls
Text-to-Speech (TTS)	Browser **SpeechSynthesis API**	Free, zero API calls

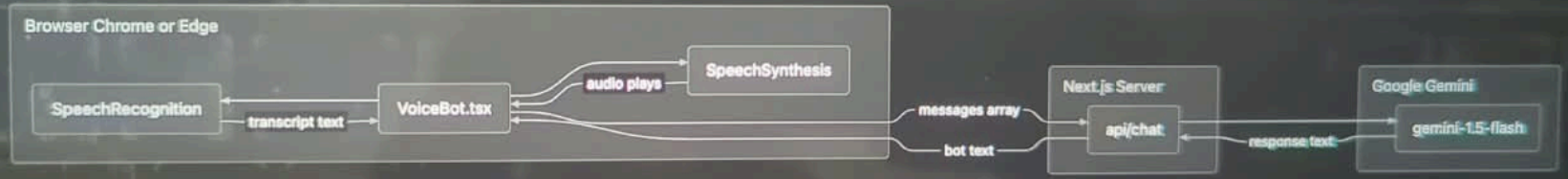
Only one API key (`GEMINI_API_KEY`) **and one server-side API route** (`/api/chat`) **are required.**

> **Browser compatibility:** Web Speech API requires Google Chrome or Microsoft Edge. It does NOT work on Firefox or Safari. Use Chrome for the demo.

Architecture



Cursor Tab Tab Stats: 0/0 (0%) Agent Stats: 0/0 (0%) No commit


Browser Chrome or Edge
SpeechRecognition
transcript text
VoiceBot.tsx
audio plays
SpeechSynthesis
messages array
bot text
Next.js Server
api/chat
response text
Google Gemini
gemini-1.5-flash

File Structure

```
snapdeal/
├── package.json
├── next.config.mjs
├── tsconfig.json
├── tailwind.config.ts
├── postcss.config.mjs
├── .env.local                    ← GEMINI_API_KEY goes here
├── app/
│   ├── globals.css
│   ├── layout.tsx
│   ├── page.tsx
│   └── api/
│       └── chat/route.ts         ← ONLY route needed: Gemini conversation
├── components/
│   └── VoiceBot.tsx              ← UI + Web Speech STT + SpeechSynthesis TTS
└── lib/
    ├── seller-data.ts            ← Mock seller data + computed deltas
    └── system-prompt.ts          ← AI persona, data context, conversation stages
```

Step 1 — Initialize Project

Run in `/Users/gauravyadav/Downloads/snapdeal`:

Step 1 — Initialize Project

Run in `/Users/gauravyadav/Downloads/snapdeal`:

```
mkdir -p app/api/chat components lib
npm init -y
npm install next@14.2.5 react@18 react-dom@18 @google/generative-ai
npm install -D typescript @types/node@20 @types/react@18 @types/react-dom@18 tailwindcss@3 autoprefixer
postcss
```

Step 2 — Config Files

`package.json` (replace entirely)

Step 2 — Config Files

`package.json` (replace entirely)

```
{
  "name": "snapdeal-voice-kam",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "@google/generative-ai": "^0.21.0",
    "next": "14.2.5",
    "react": "^18",
    "react-dom": "^18"
  },
  "devDependencies": {
    "@types/node": "^20",
    "@types/react": "^18",
    "@types/react-dom": "^18",
    "autoprefixer": "^10",
    "postcss": "^8",
    "tailwindcss": "^3",
    "typescript": "^5"
  }
}
```

`next.config.mjs`

next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = {};
export default nextConfig;
```

tsconfig.json

```
{
  "compilerOptions": {
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": { "@/*": ["./*"] }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

tailwind.config.ts

Cursor Tab ⇥ Tab Stats: 0/0 (0%) Agent Stats: 0/0 (0%) No commit scored

tailwind.config.ts

```ts
import type { Config } from 'tailwindcss';
const config: Config = {
  content: ['./app/**/*.{ts,tsx}', './components/**/*.{ts,tsx}'],
  theme: { extend: {} },
  plugins: [],
};
export default config;
```

postcss.config.mjs

```ts
const config = { plugins: { tailwindcss: {}, autoprefixer: {} } };
export default config;
```

.env.local

```
GEMINI_API_KEY=your-google-gemini-api-key-here
```

Get key from: https://aistudio.google.com/app/apikey (free, no credit card needed)

Step 3 — App Shell

app/globals.css

```
@tailwind base;
```

Step 3 — App Shell

app/globals.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
}
```

app/layout.tsx

```
import type { Metadata } from 'next';
import './globals.css';

export const metadata: Metadata = {
  title: 'Snapdeal Virtual KAM',
  description: 'AI-powered seller support voice assistant',
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

app/page.tsx

```tsx
import VoiceBot from '@/components/VoiceBot';
export default function Home() {
  return <VoiceBot />;
}
```

Step 4 — Seller Data & System Prompt

Step 4 — Seller Data & System Prompt

lib/seller-data.ts

```ts
export const SELLER_DATA = {
  name: 'Rajesh Sharma',
  store: 'Rajesh Electronics',
  category: 'Electronics',
  lastMonth: {
    sales: 200000,
    orders: 400,
    cancellationRate: 8,
    avgSellingPrice: 500,
    outOfStockProducts: 5,
  },
  thisMonth: {
    sales: 135000,
    orders: 270,
    cancellationRate: 14,
    avgSellingPrice: 500,
    outOfStockProducts: 18,
  },
};

export const COMPUTED = {
  salesDropPct: Math.round(((200000 - 135000) / 200000) * 100),  // 33%
  ordersDropPct: Math.round(((400 - 270) / 400) * 100),          // 33%
  cancellationIncrease: 14 - 8,                                  // +6 pp
  newOutOfStockCount: 18 - 5,                                    // +13 products
  lostRevenue: 200000 - 135000,                                  // ₹65,000
};
```

lib/system-prompt.ts

```
import { SELLER_DATA, COMPUTED } from './seller-data';

export function getSystemPrompt(): string {
  const d = SELLER_DATA;
  const c = COMPUTED;

  return `You are Priya, a warm and professional Virtual Key Account Manager (KAM) at Snapdeal India.
You are in an ACTIVE VOICE CALL with seller ${d.name}. These are your STRICT voice-call rules:
- Maximum 2-3 short sentences per response. NEVER more.
- NEVER use bullet points, numbered lists, or markdown formatting. Speak in natural flowing sentences.
- Sound warm, empathetic, and like a trusted advisor — not robotic or formal.
- Use Indian context: say "rupees" and reference "your Snapdeal dashboard".

SELLER PROFILE:
Name: ${d.name} | Store: ${d.store} | Category: ${d.category}

PERFORMANCE DATA (Last Month → This Month):
- Sales: ₹${d.lastMonth.sales.toLocaleString('en-IN')} → ₹${d.thisMonth.sales.toLocaleString('en-IN')}
(down ${c.salesDropPct}%)
- Orders: ${d.lastMonth.orders} → ${d.thisMonth.orders} (down ${c.ordersDropPct}%)
- Cancellation Rate: ${d.lastMonth.cancellationRate}% → ${d.thisMonth.cancellationRate}% — UP by
${c.cancellationIncrease} percentage points. RED FLAG.
- Average Selling Price: ₹${d.lastMonth.avgSellingPrice} → ₹${d.thisMonth.avgSellingPrice}. NO CHANGE —
pricing is NOT the issue.
- Out-of-Stock Products: ${d.lastMonth.outOfStockProducts} → ${d.thisMonth.outOfStockProducts} —
${c.newOutOfStockCount} MORE products out of stock. CRITICAL.

PRE-COMPUTED DIAGNOSIS (use this to guide the conversation):
1. INVENTORY CRISIS: 13 new products went out of stock this month, likely causing 100+ lost orders.
2. CANCELLATION SPIKE: Rate nearly doubled from 8% to 14%. Hurts Snapdeal search ranking via algorithm.
3. VISIBILITY TRAP: High cancellations + stockouts → Snapdeal algorithm reduces listing visibility →
fewer impressions → fewer orders. Downward spiral.
4. PRICING RULED OUT: Average selling price unchanged at ₹500, so price is not the cause.
5. LOST REVENUE: ₹${c.lostRevenue.toLocaleString('en-IN')} this month.
```

```
lostRevenue.toLocaleString('en-IN')} this month.

CONVERSATION FLOW — follow these 5 stages in sequence:

STAGE 1 — GREETING (when user says [CALL_STARTED]):
Warmly greet Rajesh by name. Acknowledge you can see changes in his account. Ask what is worrying him.
Example: "Hello Rajesh! This is Priya calling from Snapdeal. I'm really glad you reached out — I can
already see some concerning changes in your account this month. What's been worrying you the most?"

STAGE 2 — ACTIVE LISTENING (after seller explains, ask ONE targeted follow-up):
- If they mention sales drop: "I can see your orders have also fallen from 400 to 270 this month. Have
you noticed any of your products showing out of stock recently?"
- If they mention inventory: "That could be a very big factor. How many products roughly do you think
went out of stock?"
- If confused: "Let me look at your data right now. Are you getting more cancellations than usual, or
are some of your products unavailable for customers?"

STAGE 3 — DIAGNOSIS (after 1-2 follow-ups, explain the root cause clearly and empathetically):
"Rajesh, looking at your data carefully, I can see two things happening at the same time. Your out-of-
stock products have jumped from 5 to 18, and your cancellation rate has nearly doubled to 14 percent.
When products go out of stock and orders get cancelled frequently, Snapdeal's ranking algorithm
actually reduces your listing visibility — so fewer customers are even seeing your products, which
creates a downward spiral."

STAGE 4 — THREE RECOMMENDATIONS (give all three in natural speech, not a list):
First: "The most urgent action is to restock those 18 out-of-stock products within the next 48 hours —
every day they are empty, you are losing around 4 to 5 orders per product."
Second: "Next, please check the cancellation reasons in your Snapdeal seller dashboard. If it is a
logistics delay issue, we can connect you with our priority fulfillment partner to improve delivery
timelines."
Third: "Finally, I'd recommend running a 7-day promotional campaign with a 10 to 15 percent discount on
your top 5 products. This signals activity to our algorithm and helps recover your search ranking
quickly."

STAGE 5 — SUMMARY (when seller asks for recap, seems satisfied, or after recommendations have been
given):
"To summarise today's call, Rajesh: your 33 percent sales drop was primarily caused by 13 products
going out of stock and your cancellation rate climbing to 14 percent. Your three action items are:

Cursor Tab   Tab Stats: 0/0 (0%)   Agent Stats: 0/0 (0%)   No commit scored

napdeal   0 0
```

```
quickly."

STAGE 5 — SUMMARY (when seller asks for recap, seems satisfied, or after recommendations have been
given):
"To summarise today's call, Rajesh: your 33 percent sales drop was primarily caused by 13 products
going out of stock and your cancellation rate climbing to 14 percent. Your three action items are:
restock out-of-stock products in the next 48 hours, investigate the cancellation reasons in your
dashboard, and run a short promotional campaign to recover visibility. I'll also send these
recommendations to your registered Snapdeal email. Is there anything else I can help you with today?"

CLOSING (when seller says bye/thanks/that's all):
"Wonderful, Rajesh! I'm confident these steps will help you recover your sales. Don't hesitate to call
us anytime. Best of luck, and have a great day!"

IMPORTANT RULES:
- Never say more than 3 sentences in one response.
- Move the conversation forward — do not repeat what you just said.
- Be specific with numbers (say "14 percent" not "high cancellation rate").
- Never ask more than ONE question per turn.
- If seller goes off-topic, gently bring them back to the sales issue.
`;
}
```

Step 5 — API Route (only one needed)

Cursor Tab Tab Stats: 0/0 (0%) Agent Stats: 0/0 (0%) No commit scored

`app/api/chat/route.ts`

This is the only server-side file needed. STT and TTS are handled entirely in the browser for free.

Gemini uses a different message format (`role: 'model'` instead of `'assistant'`, and `parts: [{text}]` instead of `content`). The route converts between formats.

```
import { NextRequest, NextResponse } from 'next/server';
import { GoogleGenerativeAI } from '@google/generative-ai';
import { getSystemPrompt } from '@/lib/system-prompt';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);

export async function POST(req: NextRequest) {
  try {
    const { messages } = await req.json() as {
      messages: { role: 'user' | 'assistant'; content: string }[];
    };

    const model = genAI.getGenerativeModel({
      model: 'gemini-1.5-flash',      // free tier: 1500 req/day
      systemInstruction: getSystemPrompt(),
    });

    // Gemini needs all messages except the last one as "history"
    // The last message is sent as the new turn
    const history = messages.slice(0, -1).map((m) => ({
      role: m.role === 'user' ? 'user' : 'model',
      parts: [{ text: m.content }],
    }));

    const chat = model.startChat({ history });
    const lastMsg = messages[messages.length - 1];
    const result = await chat.sendMessage(lastMsg.content);
    const text = result.response.text();

    return NextResponse.json({ text });
  } catch (error) {
    console.error('[chat]', error);
    return NextResponse.json({ error: 'Chat failed' }, { status: 500 });
  }
}
```

If you have access to a newer model, change `'gemini-1.5-flash'` to `'gemini-2.0-flash'` or `'gemini-2.5-pro-preview-05-06'` on line 13. The rest of the code stays identical.

Step 6 — VoiceBot Component

The key difference from an OpenAI approach: no audio upload, no API calls for STT or TTS. Everything runs in the browser.

- `SpeechRecognition` auto-stops when the user goes silent → simpler UX (no "tap to stop" needed for STT)
- `SpeechSynthesis` converts Priya's text response to speech locally

`components/VoiceBot.tsx`

```tsx
'use client';

import { useState, useRef, useCallback, useEffect } from 'react';

type Message = { role: 'user' | 'assistant'; content: string };
type BotState = 'idle' | 'listening' | 'processing' | 'speaking' | 'error';

const TRIGGER = '[CALL_STARTED]';

export default function VoiceBot() {
  const [botState, setBotState] = useState<BotState>('idle');
  const [messages, setMessages] = useState<Message[]>([]);
  const [statusText, setStatusText] = useState('Click "Start Call" to begin');
  const [isStarted, setIsStarted] = useState(false);
  const [errorMsg, setErrorMsg] = useState<string | null>(null);

  const messagesRef = useRef<Message[]>([]);
  const bottomRef = useRef<HTMLDivElement>(null);
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  const recognitionRef = useRef<any>(null);

  // Keep ref in sync so speech recognition callbacks always read latest messages
  useEffect(() => { messagesRef.current = messages; }, [messages]);
  // Auto-scroll transcript
  useEffect(() => { bottomRef.current?.scrollIntoView({ behavior: 'smooth' }); }, [messages]);
  // Pre-load browser voices on mount (needed for SpeechSynthesis on some browsers)
  useEffect(() => {
    if (typeof window !== 'undefined') {
      window.speechSynthesis.getVoices();
      window.speechSynthesis.onvoiceschanged = () => window.speechSynthesis.getVoices();
    }
  }, []);

  /** Convert text to speech using free browser SpeechSynthesis API */
```

Cursor Tab · Tab Stats: 0/0 (0%) · Agent Stats: 0/0 (0%) · No commit scored

```
/** Convert text to speech using free browser SpeechSynthesis API */
const speakText = useCallback((text: string): Promise<void> => {
  return new Promise((resolve) => {
    setBotState('speaking');
    setStatusText('Priya is speaking…');

    window.speechSynthesis.cancel(); // stop any leftover speech

    const utterance = new SpeechSynthesisUtterance(text);
    utterance.lang = 'en-IN';
    utterance.rate = 0.92;
    utterance.pitch = 1.1;
    utterance.volume = 1.0;

    // Pick a natural-sounding female voice if available on this device
    const voices = window.speechSynthesis.getVoices();
    const femaleVoice = voices.find((v) =>
      v.lang.startsWith('en') &&
      (v.name.includes('Samantha') || v.name.includes('Victoria') ||
       v.name.includes('Karen') || v.name.toLowerCase().includes('female'))
    );
    if (femaleVoice) utterance.voice = femaleVoice;

    utterance.onend = () => {
      setBotState('idle');
      setStatusText('Your turn — tap the mic to respond');
      resolve();
    };
    utterance.onerror = () => {
      setBotState('idle');
      setStatusText('Your turn — tap the mic to respond');
      resolve();
    };

    window.speechSynthesis.speak(utterance);
  });
}, []);

/** Send full message history to Gemini via /api/chat */
```

```
/** Send full message history to Gemini via /api/chat */
const askChat = useCallback(async (msgs: Message[]): Promise<string> => {
  setBotState('processing');
  setStatusText('Priya is thinking…');
  const res = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ messages: msgs }),
  });
  if (!res.ok) throw new Error('Chat API error');
  const { text } = await res.json();
  return text as string;
}, []);

/** Start the conversation — fetch Priya's opening greeting from Gemini */
const startConversation = useCallback(async () => {
  setIsStarted(true);
  setErrorMsg(null);
  try {
    const trigger: Message = { role: 'user', content: TRIGGER };
    const botText = await askChat([trigger]);
    const greeting: Message = { role: 'assistant', content: botText };
    setMessages([trigger, greeting]); // keep trigger in history for Gemini context
    await speakText(botText);
  } catch (err) {
    console.error(err);
    setErrorMsg('Could not connect. Check GEMINI_API_KEY in .env.local and restart npm run dev.');
    setBotState('error');
    setIsStarted(false);
  }
}, [askChat, speakText]);

/** Start browser SpeechRecognition — automatically stops on silence */
```

```
  }, [askChat, speakText]);

  /** Start browser SpeechRecognition — automatically stops on silence */
  const startListening = useCallback(() => {
    setErrorMsg(null);

    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    const SR = (window as any).SpeechRecognition || (window as any).webkitSpeechRecognition;
    if (!SR) {
      setErrorMsg('Voice recognition requires Google Chrome or Microsoft Edge. Firefox is not
supported.');
      return;
    }

    const recognition = new SR();
    recognition.lang = 'en-IN';       // Indian English
    recognition.continuous = false;    // stop automatically after user finishes speaking
    recognition.interimResults = false;
    recognitionRef.current = recognition;

    recognition.onstart = () => {
      setBotState('listening');
      setStatusText('Listening… speak now');
    };

    recognition.onresult = async (event: any) => {
      const transcript = (event.results[0][0].transcript as string).trim();
      if (!transcript) return;

      const userMsg: Message = { role: 'user', content: transcript };
      const history = [...messagesRef.current, userMsg];
      setMessages(history);

      try {
        const botText = await askChat(history);
        const botMsg: Message = { role: 'assistant', content: botText };
        setMessages([...history, botMsg]);
        await speakText(botText);
      } catch (err) {
        console.error(err);
        setErrorMsg('Something went wrong. Please try again.');
        setBotState('idle');
        setStatusText('Your turn — tap the mic to respond');
      }
    };

    recognition.onerror = (event: any) => {
      console.error('SpeechRecognition error:', event.error);
      if (event.error === 'no-speech') {
        setStatusText('No speech detected. Tap the mic and try again.');
      } else if (event.error === 'not-allowed') {
        setErrorMsg('Microphone access denied. Please allow microphone for localhost in Chrome
settings.');
      }
      setBotState('idle');
    };

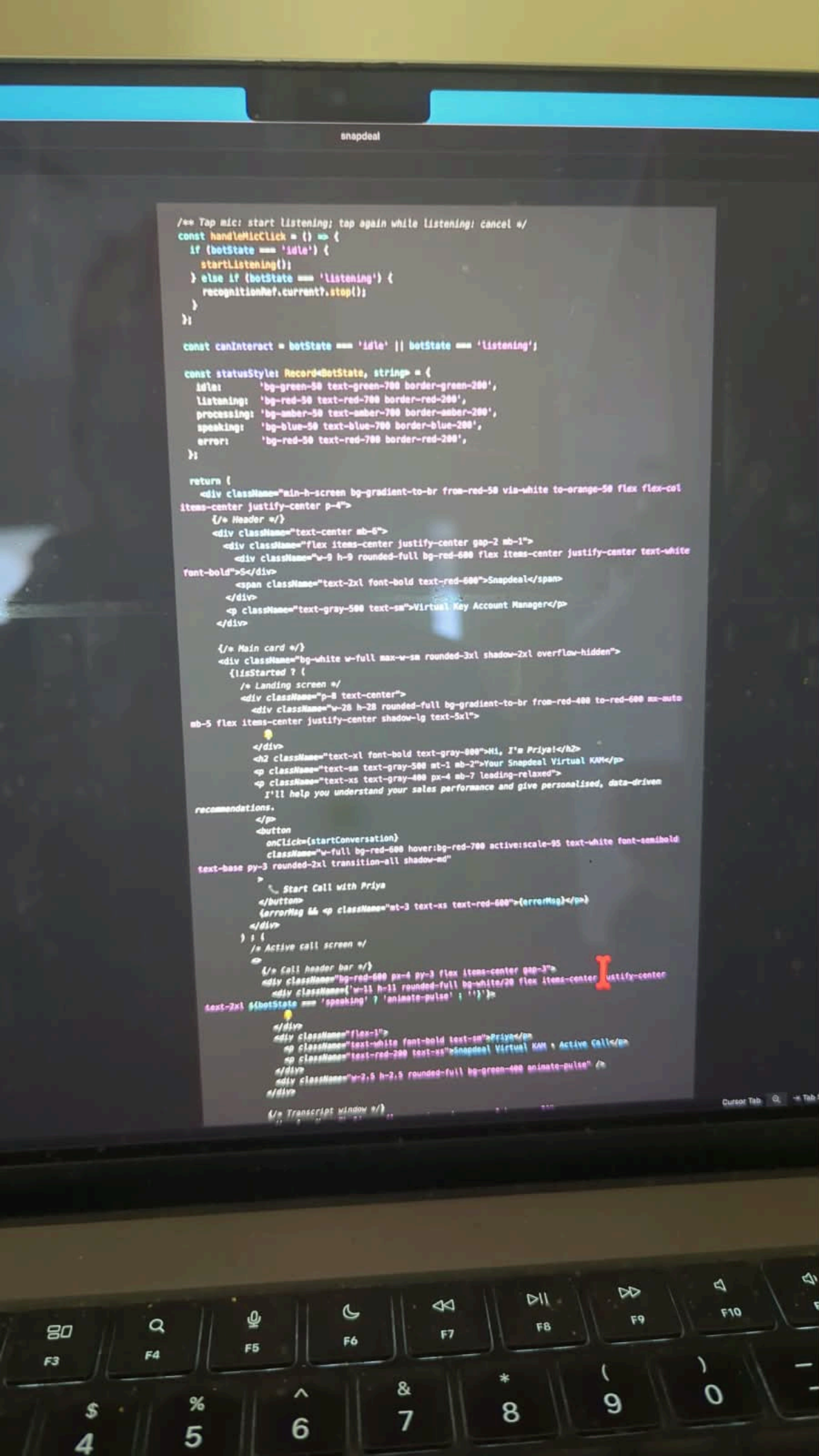
    recognition.onend = () => {
      // Only reset to idle if we are still in listening state
      // (onresult may have already moved us to processing)
      setBotState((prev) => (prev === 'listening' ? 'idle' : prev));
    };

    recognition.start();
  }, [askChat, speakText]);

  /** Tap mic: start listening; tap again while listening: cancel */
  const handleMicClick = () => {
    if (botState === 'idle') {
      startListening();
    } else if (botState === 'listening') {
      recognitionRef.current?.stop();
    }
  };
```

Cursor Tab

F4 F5 F6 F7 F8 F9 F10


snapdeal
/** Tap mic: start listening; tap again while listening: cancel */
const handleMicClick = () => {
if (botState === 'idle') {
startListening();
} else if (botState === 'listening') {
recognitionRef.current?.stop();
}
};
const canInteract = botState === 'idle' || botState === 'listening';
const statusStyle: Record<BotState, string> = {
idle: 'bg-green-50 text-green-700 border-green-200',
listening: 'bg-red-50 text-red-700 border-red-200',
processing: 'bg-amber-50 text-amber-700 border-amber-200',
speaking: 'bg-blue-50 text-blue-700 border-blue-200',
error: 'bg-red-50 text-red-700 border-red-200',
};
return (

{/* Header */}

S
Snapdeal

Virtual Key Account Manager

{/* Main card */}

{!isStarted ? (
/* Landing screen */

<h2 className="text-xl font-bold text-gray-800">Hi, I'm Priya!</h2>
Your Snapdeal Virtual KAM

I'll help you understand your sales performance and give personalised, data-driven recommendations.

<button
onClick={startConversation}
className="w-full bg-red-600 hover:bg-red-700 active:scale-95 text-white font-semibold text-base py-3 rounded-2xl transition-all shadow-md"
>
Start Call with Priya
</button>
{errorMsg && {errorMsg} }

) : (
/* Active call screen */
<>
{/* Call header bar */}

<div className={`w-11 h-11 rounded-full bg-white/20 flex items-center justify-center text-2xl ${botState === 'speaking' ? 'animate-pulse' : ''}`}>

Priya
Snapdeal Virtual KAM • Active Call

{/* Transcript window */}
Cursor Tab
F3
F4
F5
F6
F7
F8
F9
F10
4
5
6
7
8
9
0

snapdeal

```
        {/* Transcript window */}
        <div className="h-64 overflow-y-auto p-4 space-y-3 bg-gray-50">
          {messages.filter((m) => m.content !== TRIGGER).length === 0 && (
            <p className="text-center text-gray-400 text-xs mt-12">Connecting to Priya…</p>
          )}
          {messages
            .filter((m) => m.content !== TRIGGER)
            .map((msg, i) => (
              <div key={i} className={`flex ${msg.role === 'user' ? 'justify-end' : 'justify-
start'}`}>
                <div className={`max-w-[82%] rounded-2xl px-3 py-2 text-sm leading-relaxed ${
                  msg.role === 'user'
                    ? 'bg-gray-800 text-white rounded-br-sm'
                    : 'bg-white text-gray-800 shadow-sm border border-gray-100 rounded-bl-sm'
                }`}>
                  <p className="text-[10px] font-semibold opacity-60 mb-0.5">
                    {msg.role === 'user' ? 'You' : 'Priya'}
                  </p>
                  {msg.content}
                </div>
              </div>
            ))}
          <div ref={bottomRef} />
        </div>

        {/* Status bar */}
        <div className={`mx-3 my-2 rounded-xl px-4 py-2 text-xs font-medium text-center border
${statusStyle[botState]}`}>
          {botState === 'listening' && (
            <span className="inline-block w-2 h-2 rounded-full bg-red-500 mr-1.5 animate-pulse" />
          )}
          {statusText}
        </div>

        {errorMsg && (
          <p className="mx-3 mb-2 text-xs text-red-600 bg-red-50 rounded-xl px-3 py-2">{errorMsg}
</p>
        )}

        {/* Mic button */}
        <div className="flex justify-center items-center gap-4 p-3">
          <button
            onClick={handleMicClick}
            disabled={!canInteract}
            className={`w-16 h-16 rounded-full flex items-center justify-center text-2xl shadow-md
transition-all duration-150 ${
              botState === 'listening'
                ? 'bg-red-600 text-white animate-pulse scale-110'
                : botState === 'idle'
                ? 'bg-gray-900 text-white hover:bg-gray-700 active:scale-90'
                : 'bg-gray-200 text-gray-400 cursor-not-allowed'
            }`}
          >
            {botState === 'listening' ? '■' : '🎤'}
          </button>
          {botState === 'listening' && (
            <p className="text-xs text-red-600 font-semibold animate-pulse">Tap to cancel</p>
          )}
        </div>
      </>
    )}
  </div>

  <p className="mt-4 text-xs text-gray-400">
    Powered by Gemini · Web Speech API · Next.js
  </p>
</div>
  );
}
```

Step 7 — Run the App

Cursor Tab